Exercices de Compléments d'algorithmique

Série 3 – Itérateurs, Générateurs et Flux de données
(6 mars 2026)

Département d'Informatique – Faculté des Sciences – UMONS

Pré-requis : Notions d'orienté objet, itérateurs, générateurs et flux de données (cours jusqu'au **Chapitre 4**).

Objectifs : Se familiariser avec les itérateurs et générateurs en Python.

---

1 Une petite histoire pour motiver la suite

Ce TP va porter sur les générateurs, itérateurs et la gestion des flux de fichiers. Il va aussi porter une attention très particulière sur la gestion générale de la mémoire et l'optimisation de la mémoire dans votre programme. Certains d'entre-vous se disent peut-être que les contraintes mémoires ne s'appliquent que dans des domaines assez spécifiques et qu'ils n'y seront jamais confrontés. Et puis même, si un système manque de RAM, il suffit d'en ajouter, pas vrai ? Non. Pour énormément de raisons différentes (physiques, économiques, pratiques ou par contraintes des technologies actuelles), ce n'est pas toujours possible. Pour la petite histoire, un des assistants de ce cours a été confronté durant son mémoire avec le stockage d'une matrice de 255.000 entiers par 255.000 entiers. Pour votre information, cela représente environ 260GB de stockage en mémoire. Dans ce genre de cas, même écrire sur disque est la dernière alternative à envisager, même si techniquement possible.

Il faut donc réfléchir à des manières différentes de faire. Changer de structure de données (comme fait dans un TP précédent), générer les données petit à petit (comme fait dans ce TP) et filtrer les données qu'on détermine en avance comme inutiles font partie des manières les plus classiques de résoudre ces problèmes. Dans un futur proche (à l'écriture de ce TP), ce même assistant va devoir stocker une matrice de 2,5 millions par 2,5 millions d'entiers. Pour votre information, ça représente environ 25TB de stockage.

Ceci étant dit, utiliser beaucoup de mémoire permet souvent d'éviter beaucoup d'appels disque, et donc d'accélérer un calcul. Il y a donc certains cas d'utilisation où, utiliser de la mémoire libre afin de rendre une expérience plus agréable est un atout (e.g., maintenir en mémoire les fichiers souvent lus sur votre ordinateur afin d'éviter les appels disque et gagner du temps). Il y a donc souvent une balance à trouver entre la mémoire que vous consommez et la rapidité de votre programme. Cette balance dépend de votre cas d'utilisation, ainsi que des contraintes imposées à votre programme. Cependant, c'est considéré comme une bonne pratique d'optimiser sa consommation mémoire dans la mesure du possible.

Nous pouvons maintenant passer à la partie la plus fun du TP : le TP en soi-même.

2 Le retour du capitaine Edward

Lors de votre marche habituelle vers chez vous, vous entendez quelqu'un crier après vous. Alors que vous pensiez que ce n'était qu'une hallucination, que la journée était finie, vous ne voulez plus travailler, le capitaine Edward vient vous contredire en courant derrière vous. Une fois qu'il vous a rattrapé et qu'il a repris son souffle, il vous explique que sa chasse au trésor s'est légèrement complexifiée.

Votre passionné de pirate préféré vous explique que, là où avant, il n'avait que quelques points, il a maintenant trouvé une carte beaucoup plus grande avec beaucoup plus de trésors. Vous ne voyez pas encore où est le problème. Le code que vous aviez fait avant fonctionne très bien, non ? Le capitaine Edward vous explique que non, car le programme n'a pas assez de mémoire pour tourner correctement,

il s'arrête donc par manque de mémoire. Quand vous commencez à vous demander la taille de cette carte, le capitaine Edward vous explique qu'elle est stockée sous forme de matrice en fichier texte. Il vous explique qu'il a essayé de l'adapter au programme fait précédemment, mais que ça ne fonctionnait pas...

Alors que vous vous demandez qui a eu l'idée de stocker une matrice énorme dans un fichier texte, vous vous dites que tout le monde n'a pas eu l'occasion d'être à la DS Academy, et qu'il faudra donc faire avec. Il avait sûrement une très bonne raison, pas vrai ? Ceci étant dit, ça vous simplifie aussi la vie, vous pouvez la lire comme du texte.

Voyant cela comme un challenge, vous acceptez d'y regarder. En regardant un peu le fichier de la carte, vous remarquez qu'il a un format spécifique. Chaque ligne de la matrice est sur une ligne du fichier, et chaque élément de la ligne est séparé par un espace. Chaque élément représente la valeur du trésor se cachant à cet endroit. Aussi, chaque élément est toujours écrit avec 3 chiffres (3 digits). L'exemple ci-dessous résume basiquement le format du fichier. La seule différence est que le fichier qui vous est donné est nettement plus long [1].

```
000 230 331 000 954
062 845 000 000 152
000 000 963 252 000
```

Le capitaine Edward vous demande de savoir :

(1) Où est l'endroit avec la plus grande valeur du trésor ;

(2) Le nombre de trésors qu'il y a ;

(3) Enfin, la valeur moyenne de ces trésors.

Évidemment, il est nécessaire de calculer ça d'une manière propre. Ça reste la DS academy !

Vous vous dites que vous allez commencer simplement. Juste lire le fichier petit à petit sera déjà une belle avancée. Vous décidez donc de créer une classe `MatrixReader` qui s'occupera de lire la matrice petit à petit. Afin de simplifier la suite, cette classe sera itérable et vous retournera, à chaque itération, une paire $(position, nombre)$ où *nombre* sera un nombre dans la matrice et *position* représentera les coordonnées du nombre. Par exemple, pour la matrice ci-dessus, vous aurez d'abord `((0, 0), 0)`, puis `((0, 1), 230)`, suivi de `((0, 2), 331)`, etc.

1 Implémentez la classe `MatrixReader` et rendez-la itérable. Réfléchissez aux besoins de votre classe et les différentes méthodes qui vous seront utiles. Le design de la classe fait partie de l'exercice. Le fichier est formaté d'une certaine manière. Utilisez ce formatage pour vous simplifier la vie. Il est évident qu'il vous est interdit de charger l'entièreté du fichier en mémoire RAM, même si votre ordinateur vous le permet ! Notez aussi qu'une ligne entière de la matrice prend déjà beaucoup de place. Cependant, vous pouvez commencer par lire la matrice ligne par ligne, et adapter votre solution plus tard.
Conseil : Commencez par tester votre algorithme sur une plus petite matrice. Une telle matrice (10x10) peut être trouvée sur Moodle si vous le souhaitez.

Maintenant que vous savez lire le fichier sans saturer votre mémoire, nous pouvons enfin répondre au capitaine Edward.

2 Calculez le trésor de valeur maximum en itérant sur votre classe. Pour ceux qui le souhaitent, vous pouvez regarder comment utiliser la fonction `max` (implémentée par Python) dans ce but.

3 De même, calculez combien de trésors sont présents sur la carte (i.e., le nombre de cases où la valeur n'est pas nulle) et la valeur moyenne des trésors (valeurs nulles exclues). Vous ne pouvez pas stocker toutes les valeurs des trésors. Réfléchissez à une manière efficace pour votre mémoire.

Cet exercice fut éprouvant. Voulant prendre votre revanche contre le capitaine Edward pour tout ce travail effectué, vous décidez de lui faire une petite blague. Vous allez vous-même cacher la réponse dans un très long fichier. Après tout, vous avez appris à les gérer, il va apprendre avec vous s'il veut ses trésors !

1. Le principe de la carte est fort similaire à ce qui vous avez été demandé au premier TP. La principale différence est que celle-ci est nettement plus grande, contient une proportion moindre de zéros et est stockée sous un format texte.

Vous décidez de cacher la phrase « Le meilleur trésor est caché en (...) avec une valeur de ... » dans du texte aléatoire. Pour cela, vous décidez d'utiliser la notion de générateur. Vous avez pris soin de votre mémoire RAM jusqu'ici, autant continuer !

4 Créez un générateur de mots aléatoires. Ce générateur doit être infini (i.e., tant qu'on lui demande, il génère des mots). Vous pouvez utiliser le module `random` de Python, ainsi que le module `string`. Tous les caractères imprimables sont stockés dans `string.printable`. (Les mots ne doivent pas particulièrement avoir un sens en français ou en anglais.)

5 En utilisant ce générateur, écrivez dans un fichier votre réponse cachée. Choisissez aléatoirement un grand nombre de mots (disons plusieurs millions) à générer aléatoirement, utilisez votre générateur afin de les générer et écrivez-les dans votre fichier. Écrivez ensuite la phrase donnant la position du trésor et sa valeur. Enfin, choisissez à nouveau aléatoirement un grand nombre de mots que vous générerez aléatoirement et écrivez-les dans votre fichier. *Faites attention à votre consommation de mémoire RAM.*

Une fois la réponse bien cachée, vous vous demandez s'il n'y a pas d'autres trésors intéressants aux alentours. Vous décidez d'écrire une méthode afin de voir quelles sont les valeurs des trésors aux alentours de celui de plus grande valeur. La somme de tous ces trésors vous dira à quel point la zone est intéressante. Si nous considérons un carré centré sur la plus grande valeur de taille n, il nous suffit de faire la somme dans ce carré. La Figure 1 vous donne une illustration de ce carré.

1	2	3	4	5
6	7	8	9	10
11	12	999	13	14
15	16	17	18	19
20	21	22	23	24

$$7 + 8 + 9+$$
$$12 + 999 + 13+$$
$$16 + 17 + 18 = \mathbf{1099}$$

FIGURE 1 – Une petite section de la carte. Le carré de taille $n = 3$ est ici en rouge.

Pour cela, vous allez devoir accéder à certains indices de votre matrice. À nouveau, il faudrait idéalement faire ça d'une manière à préserver notre mémoire RAM et de ne pas la surcharger. Vous vous souvenez qu'il y a quand même une bonne nouvelle. Votre matrice est formatée d'une certaine manière. Vous pouvez utiliser ce formatage pour savoir calculer où se trouve (au niveau des caractères) un élément en position donnée dans votre matrice.

6 Implémentez une méthode `get_position(x, y)`. Considérant un élément en position (x, y) de votre matrice, cette méthode vous retourne la position de l'élément dans votre fichier (i.e., au quantième caractère dans votre fichier pouvez-vous trouver cet élément). Par exemple, pour $(0, 0)$, la méthode retourne 0, pour $(0, 1)$, elle retourne 4, etc.

7 De cette méthode, tirez un algorithme qui vous permet de lire les éléments en position (x, y) jusqu'à $(x + n, y)$ où n est un paramètre.
Conseil : n'oubliez pas qu'une matrice a des bords que vous risquez de dépasser.

8 Enfin, vous pouvez facilement calculer la somme des valeurs aux alentours de la case de plus grande valeur. Pour le bien du TP, considérez un carré centré sur votre plus grande valeur et de taille n. Faites varier n entre 3 et 5.

3 Du stockage d'ADN optimisé

Il y a fort fort longtemps dans un local fort fort lointain, votre professeur de théorie vous a parlé d'ADN, et comment le lire petit à petit via un script Python simple. Il vous a même donné un script

pour générer de l'ADN aléatoirement (que vous pouvez reprendre) ! Cependant, votre professeur de théorie l'a stocké sous forme de chaîne de caractères pour simplifier. Il vous avait dit qu'il y avait moyen d'optimiser un peu le stockage via une représentation binaire, mais vous n'aviez pas vraiment trop prêté attention à ce moment-là. Ce n'était qu'une parenthèse pensiez-vous. C'est donc le bon moment d'y prêter attention.

Comme vous le savez sûrement déjà, votre ordinateur associe un numéro à chaque lettre dans un texte. « a » est associé à 97, « b » à 98, etc. On réfère souvent ces numéros par l'encodage ASCII. Souvent, chaque caractère prend 1 octet (8 bits) d'espace disque afin de pouvoir représenter les lettres de l'alphabet ainsi que certains caractères spéciaux (retour à la ligne, etc.). Cependant, dans le cas de l'ADN, nous n'avons pas besoin de pouvoir représenter tout ça. Nous n'avons que 4 possibilités (A, C, G, T), et c'est tout ce dont nous avons besoin. Pour rappel, un bit peut représenter deux valeurs (0 ou 1). Si nous prenons deux bits, nous pouvons alors représenter $2^2 = 4$ valeurs (00, 01, 10 et 11). Nous pouvons, à l'instar de l'encodage ASCII, définir que chacune de ces possibilités représente une des lettres de l'ADN. Par exemple, 00 représente A, 01 représente C, etc. Donc, sur un octet (8 bits), nous pouvons en tout représenter non pas une, mais $8/2 = 4$ valeurs différentes, et ainsi, diviser la taille prise en stockant par 4, sans perdre aucune information. La Figure 2 exemplifie une représentation binaire d'ADN.

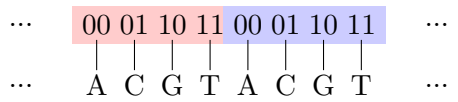


FIGURE 2 – Une représentation graphique de la représentation binaire d'ADN

Vous vous demandez sûrement comment nous pouvons accéder à uniquement deux bits dans un entier. C'est une opération binaire assez simple à effectuer via les opérateurs binaires de Python. Python implémente par défaut certains opérateurs binaires que vous pouvez appliquer. En les utilisant, il est relativement simple d'accéder à certaines parties d'un binaire. Ceci étant dit, c'est un peu hors du cadre de ce TP, mais des explications sont disponibles à la Section 5 pour les plus curieux d'entre vous. Le code faisant cela pour vous est disponible sur Moodle avec quelques exemples d'utilisations. Vous êtes cependant encouragés à lire la Section 5 et essayer de refaire ce code par vous-mêmes pour votre culture personnelle.

9 Commencez d'abord par faire un simple prototype transformant une chaîne de caractères en une liste d'octets (un entier entre 0 et 255 compris en Python) et inversement pour vous assurer d'avoir compris comment utiliser les méthodes.

10 Un code ADN est donné sur Moodle. En gardant votre mémoire RAM en tête, faites en sorte d'encoder ce fichier en fichier binaire petit à petit. Pour rappel, ce TP porte sur les itérateurs, générateurs et flux de fichiers. Utilisez-les ! Vous pouvez supposer avoir un nombre de caractères d'ADN multiple de 4 pour simplifier un peu votre vie.
Astuce : Par défaut, Python n'écrira pas des octets. Vous pouvez lui dire facilement que c'est ce que vous souhaitez en transformant votre liste d'entiers en une liste d'octets via `liste_octets = bytes(ma_liste_d_entiers)`. Aussi, vous écrivez en binaire. N'oubliez pas d'ouvrir votre fichier de manière binaire.

11 Faites l'opération inverse. D'un fichier encodé en binaire, créez un fichier encodé via des caractères.

4 Quelques exercices supplémentaires

12 Vous avez ici reçu une carte déjà créée. Vous êtes-vous demandé comment elle a été générée ? Essayer d'écrire un algorithme qui génère une carte assez grande aléatoirement en suivant la même structure que donnée ci-dessus. Faites le de deux manière : la première, garder tout en mémoire. Ensuite, générer la petit à petit. (Soyez prévenu, l'algorithme peut prendre du temps).

13 Pour la conversion de l'ADN en binaire, nous avons plus haut supposé que l'ADN avait toujours une taille multiple de 4. Voyez-vous comment nous pouvons prendre en compte les autres tailles, tout en gardant notre encodage binaire ? (Réfléchissez-y avant de passer aux exercices suivants.)

14 Une manière assez classique consiste à retenir la taille du fichier original dans le binaire. (Si nous avons 453 éléments de base, nous écrivons 453 dans le fichier binaire pour pouvoir lire facilement après le bon nombre d'éléments). Généralement, nous l'écrivons au début du fichier. Une autre manière de faire est de retenir combien de caractère nous devons ignorer à la fin, qui représente alors un nombre entre 0 et 3 (compris). En utilisant l'une de ces deux manières, adaptez votre méthode de conversion afin de pouvoir gérer des fichiers sans contrainte sur leur longueur.

5 Informations supplémentaires : accéder efficacement à une partie d'un binaire en Python

Nous allons nous intéresser à 4 opérateurs : opérateurs de shifting («, »), porte ET binaire (`&`), et la porte OU binaire (`|`).

Les opérateurs de shifting permettent de décaler un nombre binaire à gauche ou à droite. Le décalage à gauche revient à déplacer tous les bits de n places à gauche en rajoutant n 0 tout à droite. La Figure 3 montre visuellement un exemple de décalage de $n = 2$ bits à gauche.

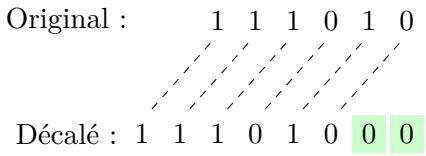


FIGURE 3 – Décalage de $n = 2$ bits à gauche pour '111010'

De manière similaire, le décalage à droite revient à déplacer tous les bits de n places à droite, en oubliant les n bits les plus à droite (les bits de poids le plus faible) et en rajoutant n 0 tout à gauche. La Figure 4 montre visuellement un exemple de décalage pour $n = 2$ bits. Remarquez que des zéros ont été ajoutés à gauche du binaire afin de décaler.

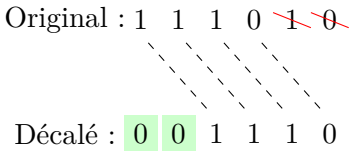


FIGURE 4 – Décalage de $n = 2$ bits à droite pour '111010'

Les opérateurs `&` et `|` sont appliqués bit à bit. La Figure 5 donne un exemple d'application. La porte OU fonctionne d'une manière similaire : `1010 | 1100` = `1110`.

BIN1 :	1	0	1	0
	&	&	&	&
BIN2 :	1	1	0	0
	=	=	=	=
& :	1	0	0	0

FIGURE 5 – Porte ET entre '1010' et '1100'

Pourquoi est-ce que cela nous est-il utile ? Si nous souhaitons accéder à la valeur des bits en position 2 à 4 d'un octet par exemple, il nous suffit de décaler l'octet afin que les bits voulus soient en position 0 à 2, et puis d'appliquer une porte ET ne gardant que ces deux derniers bits. Nous aurons alors uniquement les bits en positions 2 à 4. La Figure 6 exemplifie cette procédure.

De même, si nous souhaitons modifier des bits en positions précises dans un binaire, nous pouvons simplement utiliser les portes ET, OU et des décalages. Si nous souhaitons modifier la valeur des bits en positions 2 à 4, nous pouvons d'abord mettre leur valeur à 0 via une porte ET. Si nous appliquons la porte ET avec un binaire contenant que des 1, excepté pour des 0 en positions 2 à 4, nous aurons l'effet voulu. Ensuite, nous décalons la valeur que nous souhaitons mettre afin que les bits intéressants

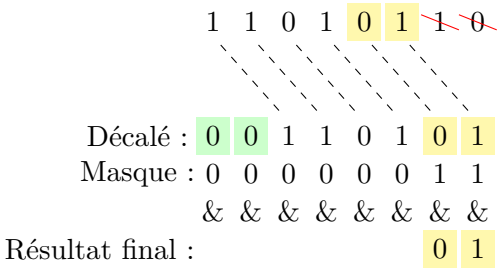


FIGURE 6 – Décalage de 2 bits à droite et application de la porte ET pour garder les 2 derniers bits

soient en position 2 à 4. Finalement, nous utilisons une porte OU afin de changer les bits en positions 2 à 4, sans affecter la valeur des autres bits. La Figure 7 exemplifie ceci.

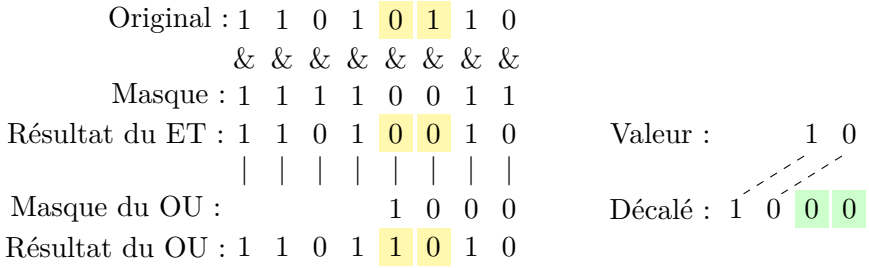


FIGURE 7 – Algorithme de modification des bits en position 2 et 3.